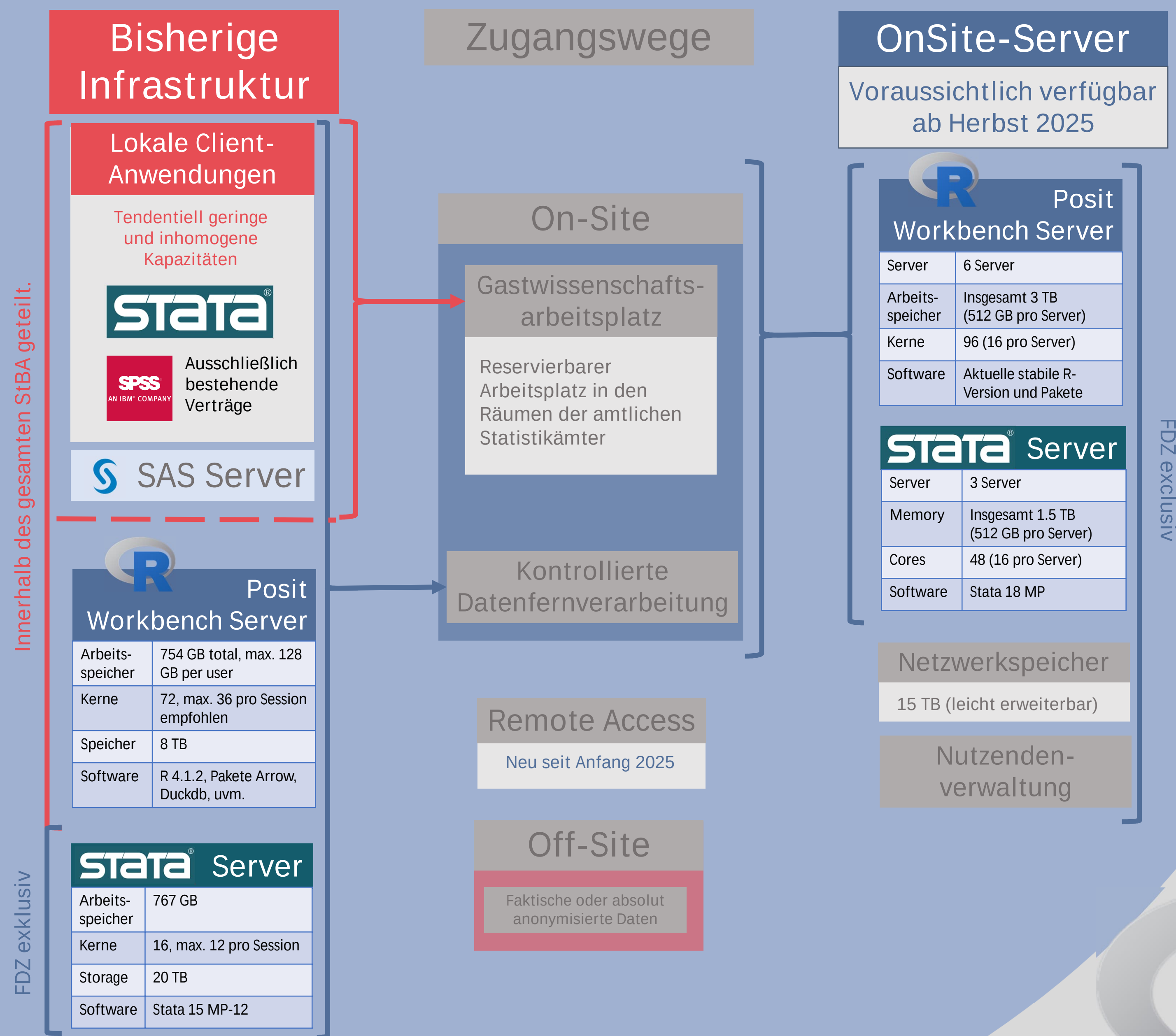


Effiziente Auswertung steuerstatistischer Daten

Potenzial der aufgebauten OnSite-Server und deren optimale Nutzung

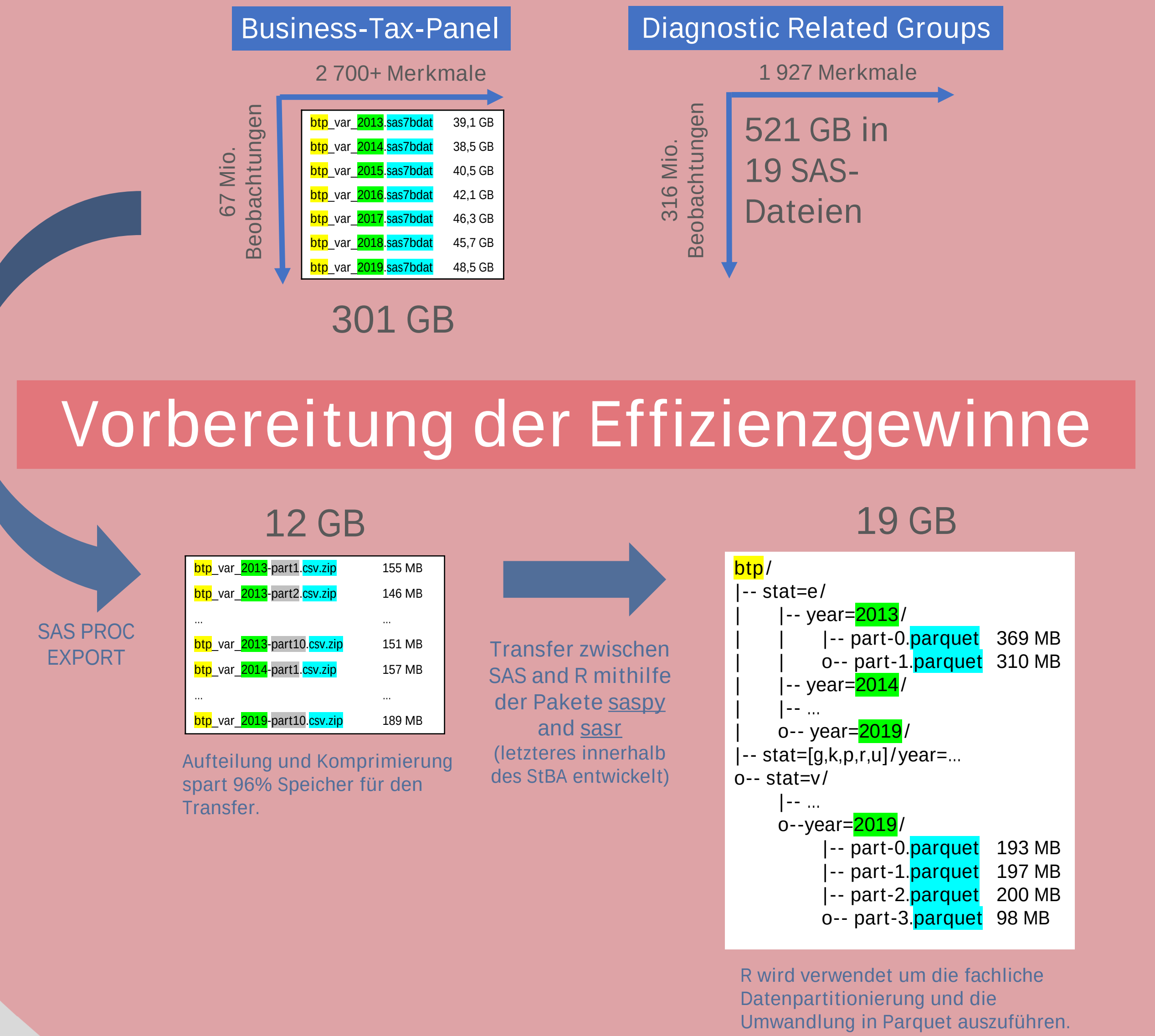
Oliver Hauke
oliver.hauke@destatis.de

Systeminfrastruktur des FDZ des Statistischen Bundesamts



Umfang des Datenangebot des FDZ

Exemplarische, ressourcen-intensive Datensätze im Angebot des FDZ sind das BTP und die DRG.



Empfehlenswerte Technologien für effiziente Softwarenutzung

Einfache Interfaces ermöglichen in R die effiziente Verarbeitung großer Datenmengen außerhalb des Arbeitsspeichers.



Apache Parquet ist ein Dateiformat, das auf das effiziente Speichern und Laden von Daten optimiert ist.

- Spalten-basiert – vorteilhaft für Analysen
- geringer Speicherbedarf durch standardmäßige Komprimierung
- Universell unterstützt in vielen Programmiersprachen

Siehe <https://parquet.apache.org>

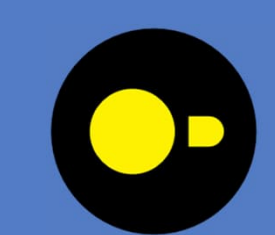


Apache Arrow ist ein spalten-basiertes Datenformat (innerhalb/außerhalb RAM) und eine Toolbox in diversen Programmiersprachen, die

- Schnelles Lesen/Schreiben (von verteilten) Daten ermöglicht
- Hoch-performante Analysen bietet.

Das R arrow package bietet das Arrow Backend durch bekannte Base und Tidyverse Funktionen.

Siehe <https://arrow.apache.org/>



DuckDB ist ein in-memory Datenbank-System, das hoch-performant komplexe Abfragen auf große Datenbanken ermöglicht.

- Anwendung ist ähnlich zu Arrow mit Fokus auf komplexere Abfragen mithilfe SQL-Syntax
- Datensätze können einfach zwischen Arrow und DuckDB ohne Performance-Verluste übergeben werden.

Siehe <https://duckdb.org>

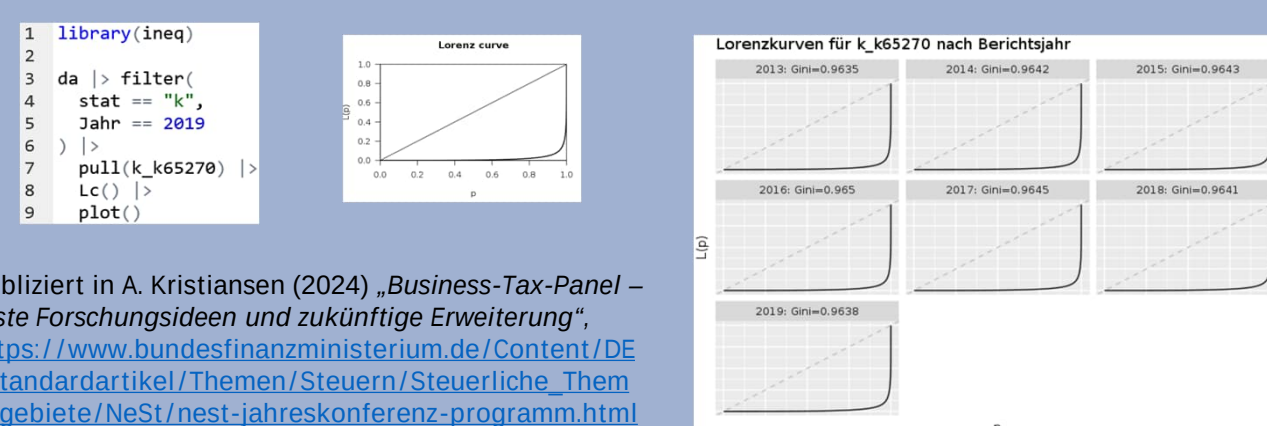
Ergebnisse der effizienten Nutzung der neuen Infrastruktur

Einlesen

```
1 library(arrow)
2 library(duckdb)
3 library(dplyr)
4 arrow::set_cpu_count(36)
5
6 source("~/data/schema_btp.R")
7 da <- open_dataset("~/data/btp",
8   schema = schema_btp)
```

- ✓ Einfach
- ✓ Verbraucht keinen RAM oder Zeit aufgrund von Lazy Computation

Verteilungsanalyse Lorenz-Kurve



- ✓ Extrem schnell – abgeschlossen in unter 2 Sekunden
 - Ein alternativer Workflow in SAS benötigt 2 Stunden
- ✓ Ermöglicht ausgeweitete Exploration und Ergebnisevaluation

Verknüpfung + Regression

Welche Merkmale haben einen Einfluss auf positive Verlustvorträge?

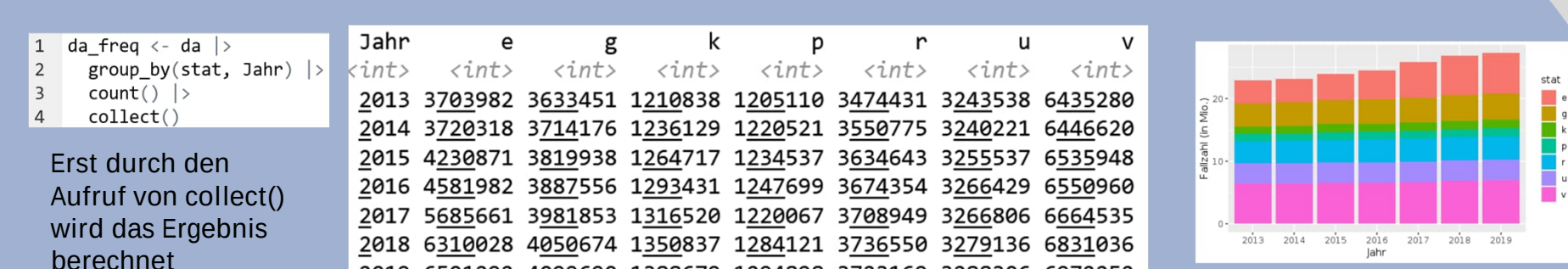
Verknüpfung von URS und KSt + Datenaufbereitung + Logit

```
fit <- glm(formula = "y ~ x", family = binomial, data = ...)
summary(fit)
```

- ✓ Extrem schnell – abgeschlossen in 18 Sekunden
- ✓ Hohe Flexibilität in Merkmalsauswahl und -verarbeitung

Deskriptive Analyse

Bspw. Fallzahlen pro Statistik und Jahr



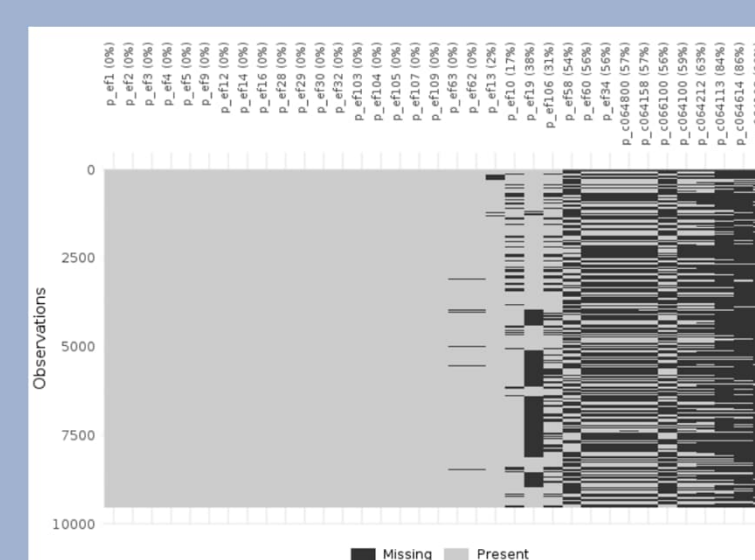
- ✓ Ermöglicht die Verarbeitung von größeren Datenmengen als der verfügbare RAM und benötigt kaum RAM (< 4 GB)

- ✓ Extrem schnell – abgeschlossen in 20 Sekunden
 - Ein alternativer SAS Workflow benötigt ca. 54 Minuten

- ✓ Einfache und gewohnte Syntax

Analyse fehlender Werte

```
1 # select subset
2 # filter(stat == "p" & jahr == 2019)
3 # starts_with("p_")
4
5 # Get variables that are less than 90% missing
6 var_low_na_sorted <- dap |
7   summarise(across(everything(),
8     ~sum(is.na(.)) < 90))
9 collect()
10 pivot_longer(everything(),
11   name_to = "variable",
12   values_to = "na")
13 filter(na == 0)
14 arrange(desc(na))
15 pull(variable)
```



- ✓ Findet geeignete Merkmale in 12 Sekunden.

Herausforderungen und Beratungsbedarf

Nicht unterstützte Operationen

Bspw. fensterweise Operationen wie `ntile()`, `paste(var1:var10)`, `c_across()`, `lm_residuais()`, `rowwise()`, ... sind nicht durch Arrow unterstützt.

Lösungsstrategien

- duckDB testen
- Alternative Befehle testen
- Pakete updaten
- select() / filter() anwenden und in-memory weiterarbeiten

Fehler, Abbrüche, unverhältnismäßige lange Laufzeiten

Bspw. bei `Join()` beobachtet

- Pakete updaten
- Verfügbaren Arbeitsspeicher prüfen
- collect() / compute() erst so spät wie möglich anwenden
- select() / filter() möglichst früh anwenden. Insb. vor join() die Merkmale/Beobachtungen reduzieren
- Mit Stichproben im Arbeitsspeicher testen

Variablentyp-Konvertierung

Speichertypen können in Arrow Fehler verursachen, bspw. integer64 können NAs erzeugen.

- Lösungsstrategien**
- Pakete updaten
 - Schema festlegen und in Parquet speichern
 - Manuelle Konvertierung vor Rechenoperationen